

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ



BÀI TIỂU LUẬN
AN TOÀN VÀ BẢO MẬT THÔNG TIN

GVHD : **Đỗ Duy Cốp**
Sinh viên thực hiện : **Trần Thị Thu Hà**
Mã số sinh viên : **K225480106009**

THÁI NGUYÊN – 2025

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ



BÀI TIỂU LUẬN
AN TOÀN VÀ BẢO MẬT THÔNG TIN

GVHD : Đỗ Duy Cốp
Sinh viên thực hiện : Trần Thị Thu Hà
Mã số sinh viên : K225480106009

THÁI NGUYÊN – 2025

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày tháng năm 2025

GIÁO VIÊN HƯỚNG DẪN CHẤM

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI NÓI ĐẦU	4
LỜI CẢM ƠN	5
CHƯƠNG 1. TỔNG QUAN DỰ ÁN	6
1.1. Lý do chọn đề tài.....	6
1.2. Mục tiêu của đề tài	6
1.3. Phạm vi nghiên cứu	6
1.4. Phương pháp thực hiện.....	7
CHƯƠNG 2. CÁC PHƯƠNG PHÁP MÃ HÓA CỔ ĐIỂN	8
2.1. Phương pháp mã hóa Caesar	8
2.1.1. Tên gọi của phương pháp mã hóa Caesar.....	8
2.1.2. Thuật toán mã hóa và giải mã.....	8
2.1.3. Không gian khóa.....	9
2.1.4. Cách phá mã mà không cần khóa	9
2.1.5. Cài đặt thuật toán mã hóa và giải mã.....	9
2.2. Phương pháp mã hóa Affine.....	11
2.2.1. Tên gọi	11
2.2.2. Thuật toán mã hóa và giải mã	11
2.2.3. Không gian khóa của thuật toán mã hóa Affine.....	12
2.2.4. Cách phá mã không cần khóa	12
2.2.5. Cài đặt thuật toán mã hóa và giải mã.....	12
2.3. Phương pháp mã hóa hoán vị.....	14
2.3.1. Tên gọi	14
2.3.2. Thuật toán mã hóa và giải mã.....	14
2.3.3. Không gian khóa.....	15
2.3.4. Cách phá mã không cần khóa	15

2.3.5. Cài đặt thuật toán mã hóa và giải mã.....	15
2.4. Phương pháp mã hóa Vigenère	17
2.4.1. Tên gọi	17
2.4.2. Thuật toán mã hóa và giải mã.....	17
2.4.3. Không gian khóa.....	17
2.4.4. Cách phá mã không cần khóa	17
2.4.5. Cài đặt thuật toán mã hóa và giải mã.....	17
2.5. Phương pháp mã hóa PlayFair	19
2.5.1. Tên gọi	19
2.5.2. Thuật toán mã hóa và giải mã.....	19
2.5.3. Không gian khóa.....	20
2.5.4. Cách phá mã không cần khóa	20
2.5.5. Cài đặt thuật toán mã hóa và giải mã.....	20
CHƯƠNG 3. CHỮ KÝ SỐ TRONG FILE PDF	23
3.1. Cấu trúc file PDF liên quan chữ ký.	23
3.2. Nơi lưu thời gian ký.....	25
3.3. Các rủi ro bảo mật trong chữ ký số PDF	27
3.4. Quy trình tạo và lưu chữ ký số trong file PDF	28
3.4.1. Chuẩn bị một file PDF gốc cần ký	30
3.4.2. Tạo file sinh khóa RSA và chứng thư số	30
3.4.3. Tạo chữ ký và ký file PDF	30
3.5. Xác thực chữ ký trên file PDF đã ký	32
3.6. Kết quả demo	33
TÀI LIỆU THAM KHẢO.....	35
LINK GITHUB.....	36

LỜI NÓI ĐẦU

Trong thời đại số hóa hiện nay, đảm bảo an toàn và bảo mật thông tin đóng vai trò vô cùng quan trọng trong mọi hệ thống công nghệ. Từ những phương pháp mã hóa cổ điển xuất hiện hàng trăm năm trước cho đến các kỹ thuật chữ ký số hiện đại, mục tiêu chung đều nhằm bảo vệ dữ liệu khỏi sự truy cập trái phép và đảm bảo tính toàn vẹn của thông tin.

Bài tiểu luận này tập trung tìm hiểu các loại **mã hóa cổ điển** như Caesar, Affine, hoán vị, Playfair và Vigenère – những nền tảng cơ bản giúp hình thành các kỹ thuật mật mã hiện đại. Bên cạnh đó, bài viết cũng nghiên cứu về **chữ ký số trong tệp PDF**, bao gồm quy trình ký, cách xác thực và đặc biệt là phương pháp thay đổi nội dung văn bản mà không ảnh hưởng đến tính hợp lệ của chữ ký.

Thông qua việc nghiên cứu hai nhóm nội dung trên, em mong muốn góp phần làm rõ nguyên lý hoạt động, ưu nhược điểm và ứng dụng của các kỹ thuật bảo mật, từ đó nâng cao hiểu biết và ý thức trong việc bảo vệ thông tin.

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến giáo viên hướng dẫn Đỗ Duy Cốp đã tận tình giảng dạy, truyền đạt những kiến thức bổ ích và định hướng quý báu cho chúng em trong suốt quá trình học tập. Chính những nền tảng lý thuyết và kinh nghiệm thực tiễn mà thầy đã chia sẻ là nguồn động lực quan trọng để em có thể hoàn thành bài tiểu luận này.

Em cũng xin gửi lời cảm ơn đến các bạn trong lớp đã đồng hành, trao đổi, chia sẻ tài liệu và hỗ trợ em trong quá trình tìm hiểu, phân tích và triển khai bài làm. Mặc dù đã có nhiều cố gắng, nhưng do kiến thức và kinh nghiệm còn hạn chế, bài làm chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp của thầy cô và các bạn để bài làm có thể hoàn thiện hơn.

Một lần nữa, em xin chân thành cảm ơn!

CHƯƠNG 1. TỔNG QUAN DỰ ÁN

1.1. Lý do chọn đề tài

Trong bối cảnh công nghệ ngày càng phát triển mạnh mẽ, thông tin trở thành tài sản quan trọng cần được bảo vệ trước những nguy cơ tấn công mạng, đánh cắp dữ liệu và giả mạo tài liệu. Các kỹ thuật mật mã từ cổ điển đến hiện đại đều có vai trò quan trọng trong việc đảm bảo tính bí mật, tính toàn vẹn và tính xác thực của thông tin.

Mặc dù mật mã hiện đại đã rất phát triển, nhưng các thuật toán mã hóa cổ điển vẫn có giá trị lớn về mặt lịch sử, giúp hiểu rõ nguyên lý hình thành và phát triển của ngành mật mã. Bên cạnh đó, chữ ký số - đặc biệt là chữ ký số trong tệp PDF – là ứng dụng phổ biến trong thực tế, giúp hỗ trợ xác thực tài liệu điện tử trong hành chính, doanh nghiệp và giao dịch trực tuyến.

Vì vậy bài tiểu luận này sẽ được thực hiện kết hợp giữa mã hóa cổ điển và chữ ký số trong PDF, cung cấp cái nhìn tổng quan và thực tiễn trong lĩnh vực an toàn và bảo mật thông tin.

1.2. Mục tiêu của đề tài

Đề tài hướng đến các mục tiêu chính sau:

- Tìm hiểu nguyên lý hoạt động của các phương pháp **mã hóa cổ điển** gồm: Caesar, Affine, hoán vị, Playfair và Vigenère.
- Phân tích ưu điểm, nhược điểm và khả năng tấn công đối với từng thuật toán.
- Tìm hiểu cơ chế **chữ ký số trong tệp PDF**, bao gồm cấu trúc chữ ký, quy trình ký và xác thực.
- Phân tích cách thức **thay đổi nội dung văn bản PDF mà không làm mất hiệu lực chữ ký**, thông qua cơ chế ByteRange và vùng Contents.
- Hệ thống hóa kiến thức nhằm củng cố nền tảng bảo mật thông tin.

1.3. Phạm vi nghiên cứu

Nội dung nghiên cứu của bài tiểu luận này sẽ tập trung vào:

- Các kỹ thuật mã hóa cổ điển mức cơ bản đến trung cấp, bao gồm cách mã hóa và giải mã và ví dụ minh họa.
- Chữ ký số sử dụng chuẩn CMS/PKCS#7 trong tệp PDF.
- Cấu trúc file PDF liên quan đến chữ ký số, đặc biệt là **/ByteRange**, **/Contents** và **Signature Dictionary**.

1.4. Phương pháp thực hiện

Đề tài này được thực hiện dựa trên các phương pháp:

- Nghiên cứu lý thuyết: Thu thập và tổng hợp tài liệu từ giáo trình, tiêu chuẩn PDF và tài liệu mật mã học.
- Thử nghiệm thực hành: mã hóa – giải mã thử nghiệm các thuật toán cổ điển, thử nghiệm ký và xác thực chữ ký trên file PDF.
- Phân tích – đối chiếu: So sánh các phương pháp sau đó rút ra nhận xét và kết luận.

CHƯƠNG 2. CÁC PHƯƠNG PHÁP MÃ HÓA CỔ ĐIỂN

2.1. Phương pháp mã hóa Caesar

2.1.1. Tên gọi của phương pháp mã hóa Caesar

Mật mã Caesar hay còn gọi là mật mã chuyển vị là một trong những kỹ thuật mã hóa đơn giản và phổ biến nhất. Đây là một dạng mật mã thay thế, trong mỗi ký tự trên bản rõ sẽ được thay bằng một ký tự khác có vị trí cách nó một khoảng xác định trong bảng chữ cái (khoảng này được gọi là khóa K). Tên của kỹ thuật mã hóa này được đặt theo tên của Julius Caesar, người đã sử dụng phương pháp mã hóa này trong các thư từ bí mật của mình.

2.1.2. Thuật toán mã hóa và giải mã

Thuật toán mã hóa:

- Ta sẽ sử dụng bảng chữ cái có 26 ký tự từ A đến Z và mỗi chữ cái sẽ gán tuần tự với một số nguyên tương ứng từ 0 đến 25 (A->0, B->1,... Z->25). Mã hóa một chữ cái x bằng cách dịch chuyển n vị trí được mô tả bằng phương thức toán học sau:

$$Ek(x) = (x + k) \text{ mode } 26$$

Trong đó:

- + k là khóa – độ dịch chuyển
- + x là bản rõ – ký tự cần mã hóa
- + E là bản mã

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

Hình 2.1: Bảng chữ cái mã hóa

Thuật toán giải mã:

- Thuật toán giải mã sẽ ngược lại với thuật toán mã hóa, nó sẽ được thực hiện như sau:

$$Dk(x) = (x - k) \text{ mode } 26$$

Trong đó:

- + D là bản rõ
- + x là bản mã – ký tự cần giải mã
- + k là khóa – độ dịch chuyển

2.1.3. Không gian khóa

- Nếu dùng bảng 26 chữ cái thì số khóa là 26 khóa ($k = 25$)
 - Nhưng $k = 0$ và $k = 26$ cho cùng một vị trí hoán vị nên thực tế chỉ có 25 khóa hữu dụng nếu bỏ $k = 0$ (không dịch).
- ⇒ Không gian khóa của thuật toán mã hóa Caesar rất nhỏ và dễ bị tấn công

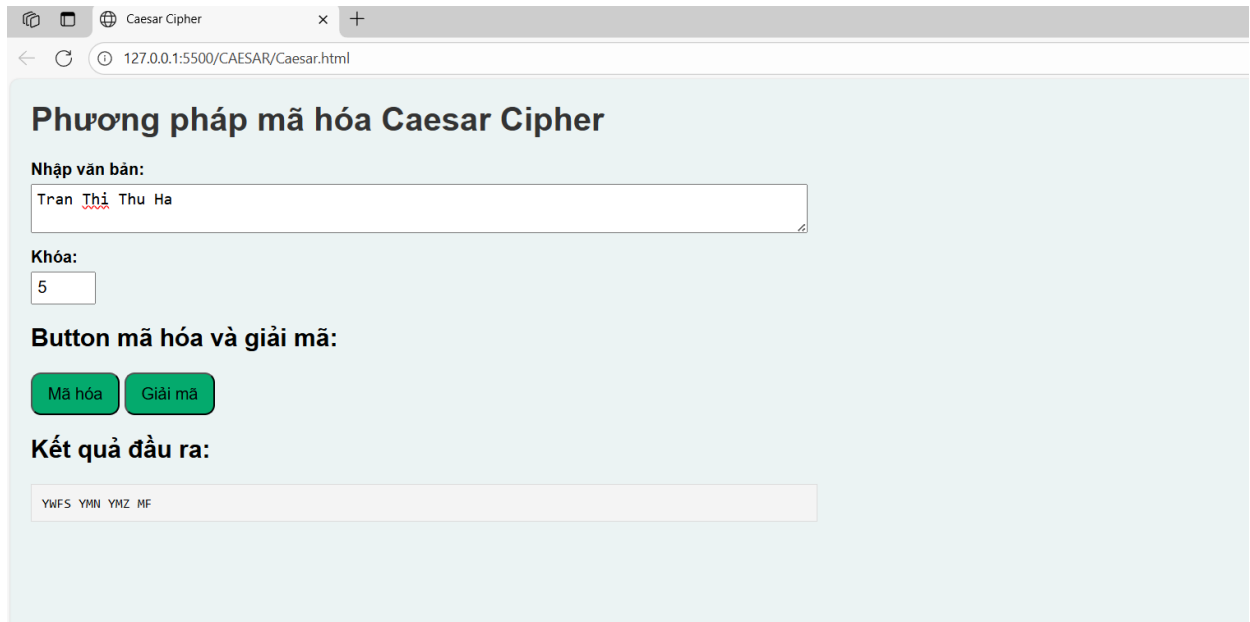
2.1.4. Cách phá mã mà không cần khóa

Do không gian khóa của thuật toán này rất nhỏ nên nó dễ bị tấn công bằng nhiều cách như:

- **Tấn công vét cạn (Brute-force):** Cách này sẽ thử tất cả 26 khả năng và đọc kết quả.
- **Phân tích tần suất:** Trong các văn bản dài thì tần suất xuất hiện của các chữ cái cố định sẽ nhiều, vì thế mà các hacker có thể so sánh chữ cái phổ biến nhất trong đoạn mã từ đó suy đoán ra chữ cái nào đã bị thay thế và tìm ra khóa k.
- **Known - plaintext / crib:** Phương pháp này là nếu biết một đoạn nhỏ của bản rõ thì có thể suy ra khóa k ngay.

2.1.5. Cài đặt thuật toán mã hóa và giải mã

Mã hóa bằng HTML + CSS + Javascript:



Phương pháp mã hóa Caesar Cipher

Nhập văn bản:

Tran Thi Thu Ha

Khóa:

5

Button mã hóa và giải mã:

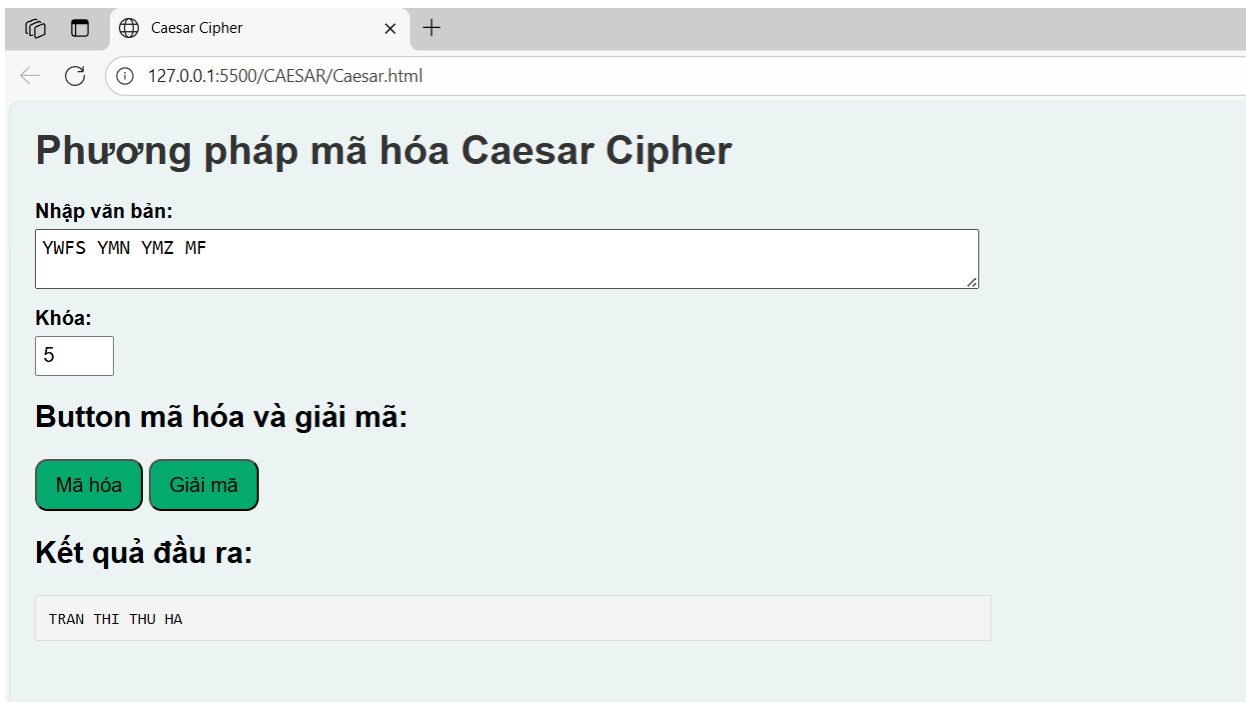
Mã hóa Giải mã

Kết quả đầu ra:

YWFS YMN YMZ MF

Hình 2.2. Mã hóa Caesar

Giải mã bằng HTML + CSS + Javascript:



Phương pháp mã hóa Caesar Cipher

Nhập văn bản:

YWFS YMN YMZ MF

Khóa:

5

Button mã hóa và giải mã:

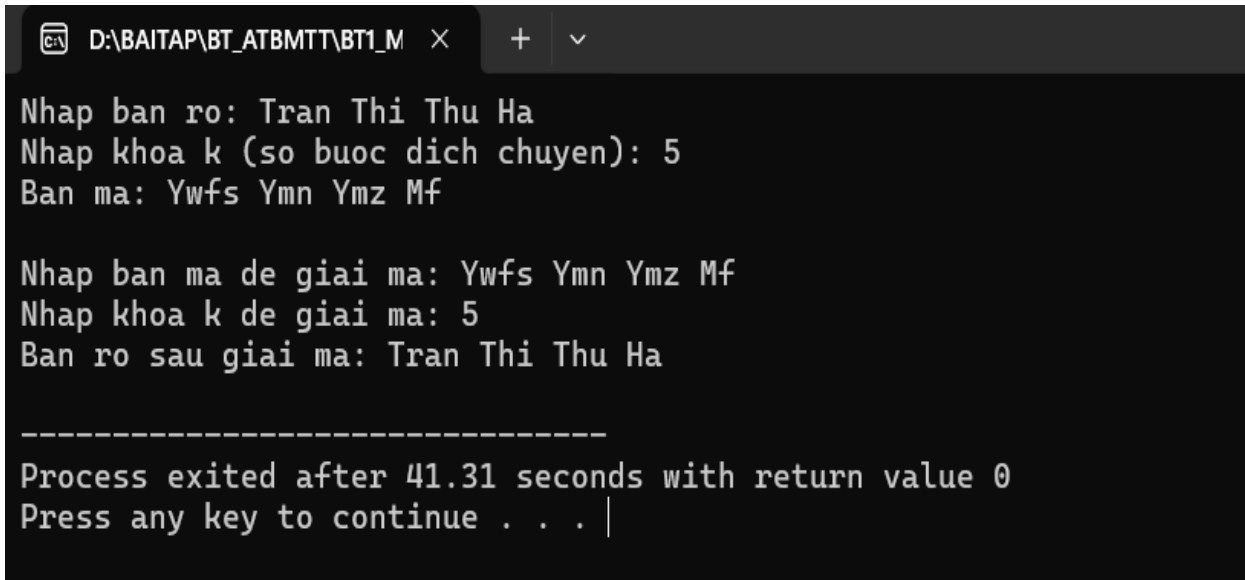
Mã hóa Giải mã

Kết quả đầu ra:

TRAN THI THU HA

Hình 2.3. Giải mã thuật toán Caesar

Mã hóa và giải mã bằng C++:



```
D:\BAITAP\BT_ATBMTT\BT1_M x + v
Nhap ban ro: Tran Thi Thu Ha
Nhap khoa k (so buoc dịch chuyen): 5
Ban ma: Ywfs Ymn Ymz Mf

Nhap ban ma de giai ma: Ywfs Ymn Ymz Mf
Nhap khoa k de giai ma: 5
Ban ro sau giai ma: Tran Thi Thu Ha

-----
Process exited after 41.31 seconds with return value 0
Press any key to continue . . . |
```

Hình 2.4. Mã hóa và giải mã Caesar bằng C++

2.2. Phương pháp mã hóa Affine

2.2.1. Tên gọi

Mật mã Affine là một dạng mật mã thay thế dùng một bảng chữ cái trong đó mỗi chữ cái được ánh xạ tới một số, sử dụng hàm tuyến tính trong số học module.

2.2.2. Thuật toán mã hóa và giải mã

Thuật toán mã hóa:

- Biểu diễn mỗi chữ cái bằng số từ 0 đến 25 ($A = 0, B = 1, \dots, Z = 25$).
- Dùng hàm mã hóa: $E(x) = (a \cdot x + b) \bmod 26$

Trong đó:

- + x là giá trị số của chữ cái bản rõ.
- + a, b là hai số nguyên tạo thành khóa
- + a phải thỏa mãn điều kiện: $\gcd(a, 26) = 1$ (a và 26 phải là nguyên tố cùng nhau) để đảm bảo tồn tại nghịch đảo.

Thuật toán giải mã:

- Dùng hàm giải mã: $D(y) = a^{-1} \cdot (y - b) \text{ mode } 26$

Trong đó:

+ y là giá trị số của chữ cái bản mã.

+ a^{-1} là nghịch đảo của a theo module 26 (là $a \cdot a^{-1} \equiv 1 \text{ mode } 26$).

2.2.3. Không gian khóa của thuật toán mã hóa Affine

Không gian khóa của phương pháp mã hóa Affine có khóa gồm cặp số (a, b) với:

- $a \in \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$ (12 giá trị thỏa mãn $\gcd(a, 26) = 1$).
- $b \in \{0, 1, \dots, 25\}$.

$\Rightarrow$ Có tổng 213 khóa (12×26).

2.2.4. Cách phá mã không cần khóa

- **Phương pháp vét cạn (Brute-force):** Thử tất cả 312 cặp khóa (a, b) sau đó giải mã bản mã.
- **Phân tích tần suất:** So sánh tần suất xuất hiện của các chữ cái trong bản mã với tần suất chuẩn của bản rõ $\Rightarrow$ Suy đoán các chữ cái bị thay thế và tìm ra khóa.

2.2.5. Cài đặt thuật toán mã hóa và giải mã

Mã hóa Affine bằng HTML + CSS + Javascript:

A screenshot of a web browser showing a file named 'Affine Cipher.html'. The page title is 'Mã hóa Affine Cipher'. It features a text input field with the value 'Tran Thi Thu Ha KMT'. Below it, there are two input fields for keys 'a' (15) and 'b' (10). Two green buttons, 'Mã hóa' and 'Giải mã', are present. The 'Kết quả:' section displays the encrypted text 'JFKX JLA JLY LK EIJ'.

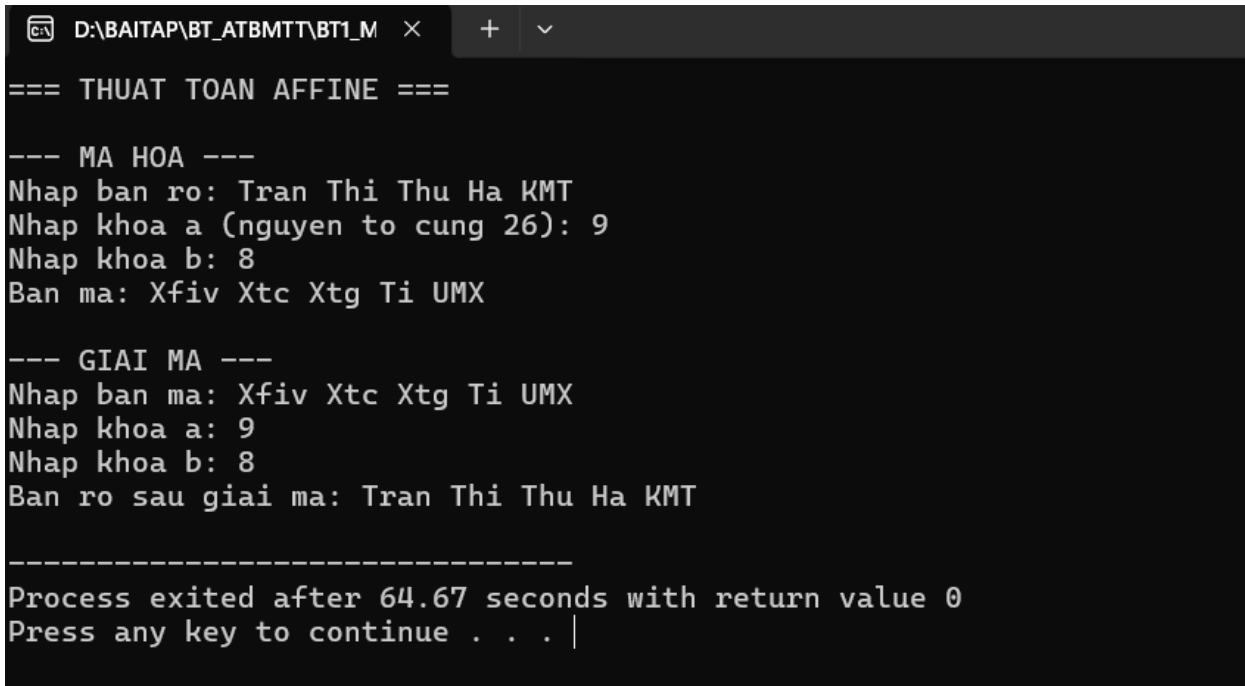
Hình 2.5. Mã hóa Affine

Giải mã Affine bằng HTML + CSS + Javascript:

A screenshot of the same web browser showing the 'Affine Cipher.html' file. The page title is 'Mã hóa Affine Cipher'. The text input field now contains the encrypted text 'JFKX JLA JLY LK EIJ'. The key inputs for 'a' (15) and 'b' (10) remain the same. The 'Mã hóa' and 'Giải mã' buttons are still visible. The 'Kết quả:' section now displays the decrypted text 'TRAN THI THU HA KMT'.

Hình 2.6. Giải mã Affine

Mã hóa và giải mã Affine bằng C++:



```
=== THUAT TOAN AFFINE ===

--- MA HOA ---
Nhap ban ro: Tran Thi Thu Ha KMT
Nhap khoa a (nguyen to cung 26): 9
Nhap khoa b: 8
Ban ma: Xfiv Xtc Xtg Ti UMX

--- GIAI MA ---
Nhap ban ma: Xfiv Xtc Xtg Ti UMX
Nhap khoa a: 9
Nhap khoa b: 8
Ban ro sau giai ma: Tran Thi Thu Ha KMT

-----
Process exited after 64.67 seconds with return value 0
Press any key to continue . . . |
```

Hình 2.7. Mã hóa và giải mã Affine bằng C++

2.3. Phương pháp mã hóa hoán vị

2.3.1. Tên gọi

Mã hóa hoán vị

2.3.2. Thuật toán mã hóa và giải mã

Nguyên lý:

- Chia văn bản thành các block có độ dài n (n = độ dài khóa / hoán vị).
- Với mỗi block, tái sắp xếp các ký tự theo một hoán vị P
- Nếu block cuối không đủ n ký tự thì pad (ví dụ 'x')

Mã hóa: Với mỗi block B ($b_0..b_{n-1}$), tạo ciphertext block C sao cho $C[i] = B[P[i]]$

Giải mã: Dùng hoán vị nghịch đảo P^{-1} để đặt các ký tự về vị trí ban đầu:

$$B[i] = C[P^{-1}[i]]$$

2.3.3. Không gian khóa

Nếu khóa là một hoán vị có độ dài n thì số lượng khóa sẽ $= n!$, nếu n nhỏ thì $n!$ cũng sẽ nhỏ và dễ bị tấn công vét cạn. Nếu n lớn thì $n!$ cũng lớn, không gian khóa rất to.

2.3.4. Cách phá mã không cần khóa

- **Vét cạn (Brute-force):** Thử tất cả các hoán vị có thể.
- **Phân tích cấu trúc:** Dựa vào độ dài và tần suất xuất hiện của các ký tự để đoán.

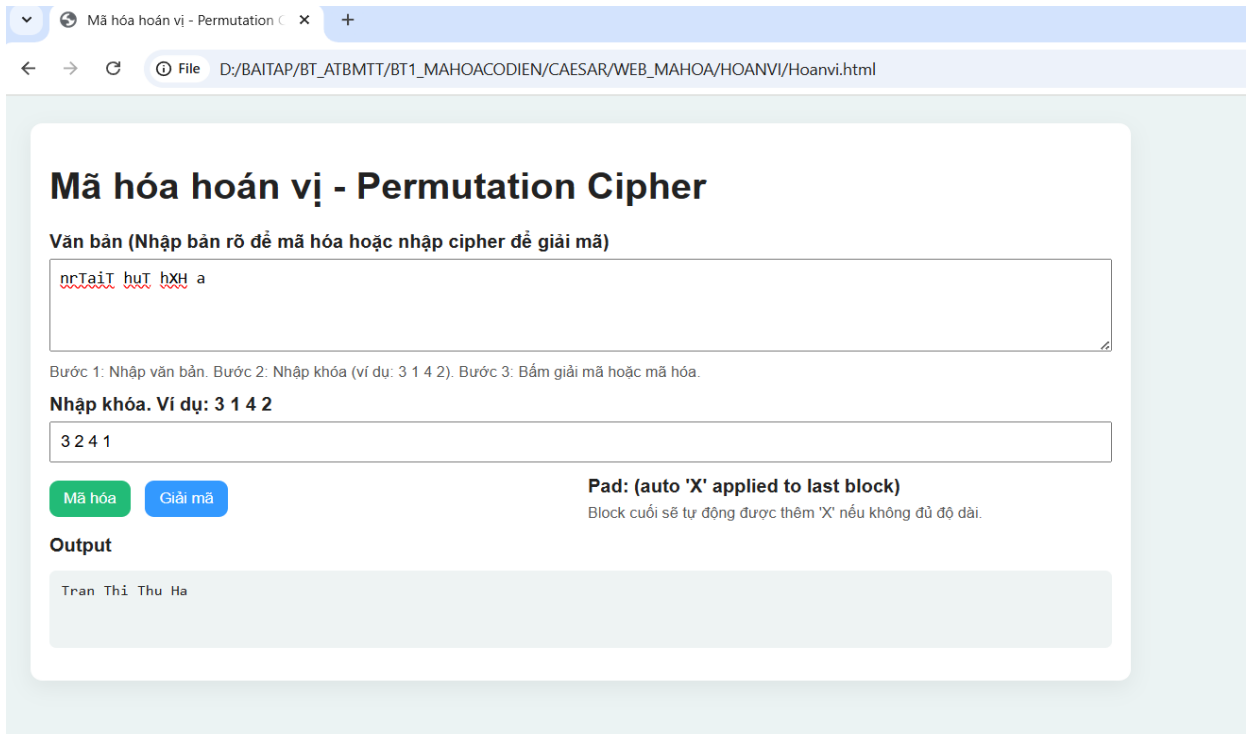
2.3.5. Cài đặt thuật toán mã hóa và giải mã

Mã hóa bằng HTML + CSS + Javascript:

The screenshot shows a web browser window with the address bar displaying the file path: D:\BAITAP\BT_ATBMTT\BT1_MAHOCODIEN\CAESAR\WEB_MAHOA\HOANVI\Hoanvi.html. The page title is 'Mã hóa hoán vị - Permutation Cipher'. The main content area has a heading 'Mã hóa hoán vị - Permutation Cipher' and a sub-heading 'Văn bản (Nhập bản rõ để mã hóa hoặc nhập cipher để giải mã)'. Below this is a text input field containing 'Tran Thi Thu Ha'. A small instruction below the input field reads: 'Bước 1: Nhập văn bản. Bước 2: Nhập khóa (ví dụ: 3 1 4 2). Bước 3: Bấm giải mã hoặc mã hóa.' Below this is a section for 'Nhập khóa. Ví dụ: 3 1 4 2' with a text input field containing '3 2 4 1'. There are two buttons: 'Mã hóa' (green) and 'Giải mã' (blue). To the right of these buttons is a note: 'Pad: (auto 'X' applied to last block)' and 'Block cuối sẽ tự động được thêm 'X' nếu không đủ độ dài.' Below the buttons is an 'Output' section with a text area displaying the encrypted message: 'nrTa1T huT hXH a'.

Hình 2.8. Mã hóa hoán vị

Giải mã bằng HTML + CSS + Javascript:



Hình 2.9. Giải mã thuật toán mã hóa hoán vị

Mã hóa và giải mã bằng C++:

```
D:\BAITAP\BT_ATBMTT\BT1_M x + v
===== CHUONG TRINH MA HOA HOAN VI =====
1. Ma hoa
2. Giai ma
3. Thoat
Chon: 1
Nhap ban ro: TRAN THI THU HA
Nhap do dai khoa: 4
Nhap khoa (vi tri tu 1 den 4): 3 1 2 4
Ban ma: ATRNH TIH TUA HX

===== CHUONG TRINH MA HOA HOAN VI =====
1. Ma hoa
2. Giai ma
3. Thoat
Chon: 2
Nhap ban ma: ATRNH TIH TUA HX
Nhap do dai khoa: 4
Nhap khoa (vi tri tu 1 den 4): 3 1 2 4
Ban giai ma: TRAN THI THU HA

===== CHUONG TRINH MA HOA HOAN VI =====
1. Ma hoa
2. Giai ma
3. Thoat
Chon: 3
Thoat chuong trinh.

-----
Process exited after 31.53 seconds with return value 0
Press any key to continue . . . |
```

Hình 2.10. Mã hóa và giải mã thuật toán mã hóa hoán vị bằng C++

2.4. Phương pháp mã hóa Vigenère

2.4.1. Tên gọi

Mã hóa Vigenere cipher

2.4.2. Thuật toán mã hóa và giải mã

Thuật toán mã hóa:

- Biểu diễn mỗi chữ cái bằng số từ 0 đến 25 ($A = 0, B = 1, \dots, Z = 25$).
- Công thức: $C_i = (P_i + K_i) \text{ mode } 26$

Trong đó:

- + P là bản rõ
- + K là khóa
- + C là bản mã

Thuật toán giải mã:

- Thuật toán giải mã của phương pháp mã hóa Vigenere sẽ được thực hiện theo công thức: $P_i = (C_i - K_i + 26) \text{ mode } 26$

2.4.3. Không gian khóa

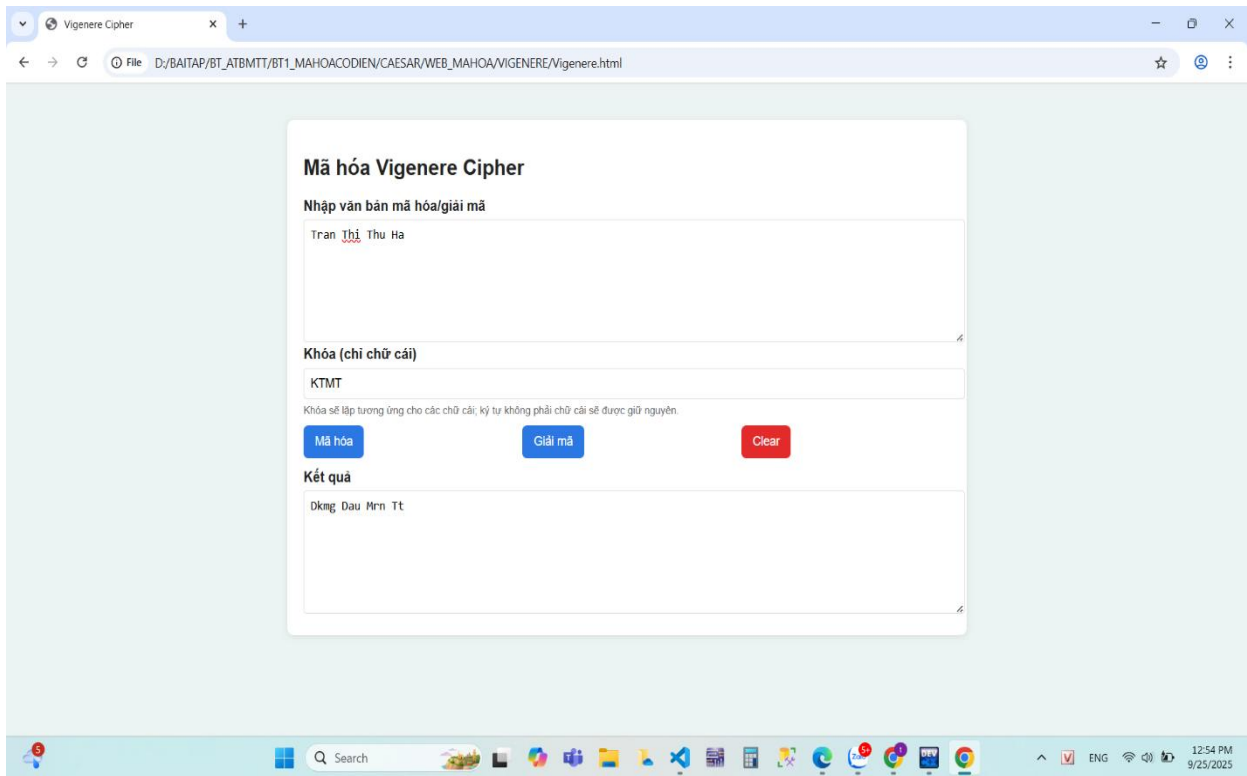
Nếu là key dài m (m chữ cái, mỗi chữ có 26 khả năng), thì không gian khóa là 26^m . Nếu key có độ dài thay đổi thì không gian khóa sẽ lớn và phức tạp hơn.

2.4.4. Cách phá mã không cần khóa

- **Phân tích tần suất:** Phân chia bản mã thành các nhóm theo chu kỳ khóa để tìm độ dài khóa
- **Kasiski examination:** Tìm các chuỗi lặp trong ciphertext; khoảng cách giữa các lần lặp thường chia hết cho độ dài khóa $\rightarrow$ suy ra độ dài khóa.
- **Friedman test:** Tính chỉ số trùng lặp.

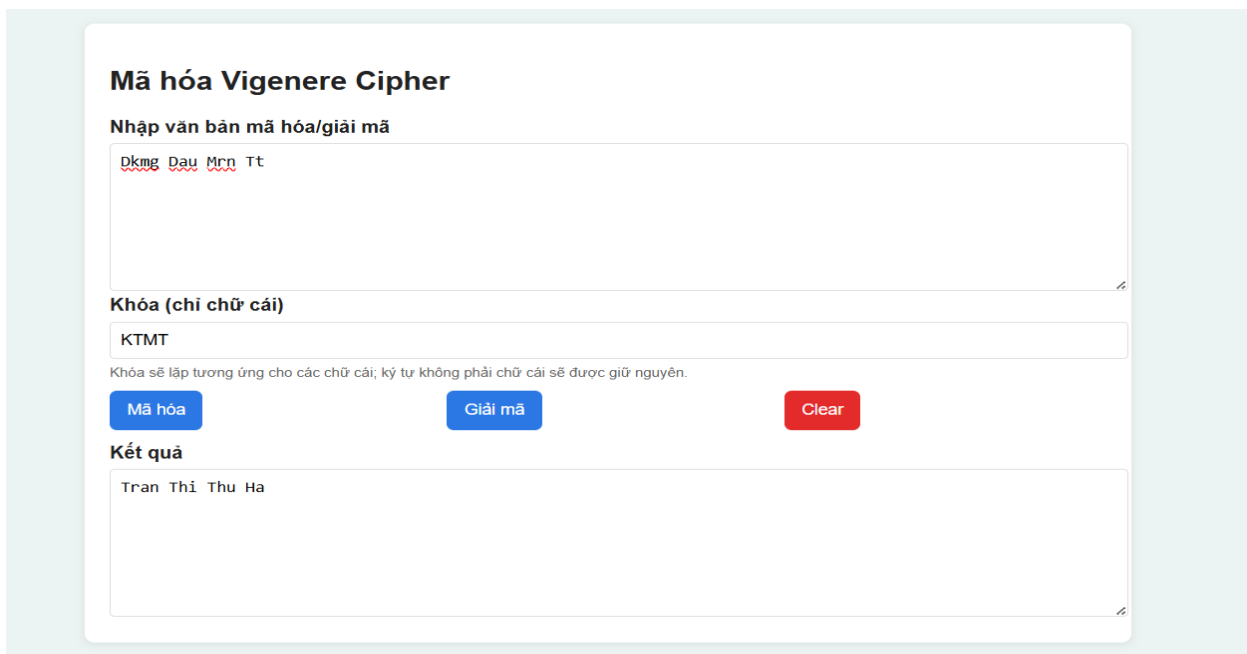
2.4.5. Cài đặt thuật toán mã hóa và giải mã

Mã hóa bằng HTML + CSS + Javascript:



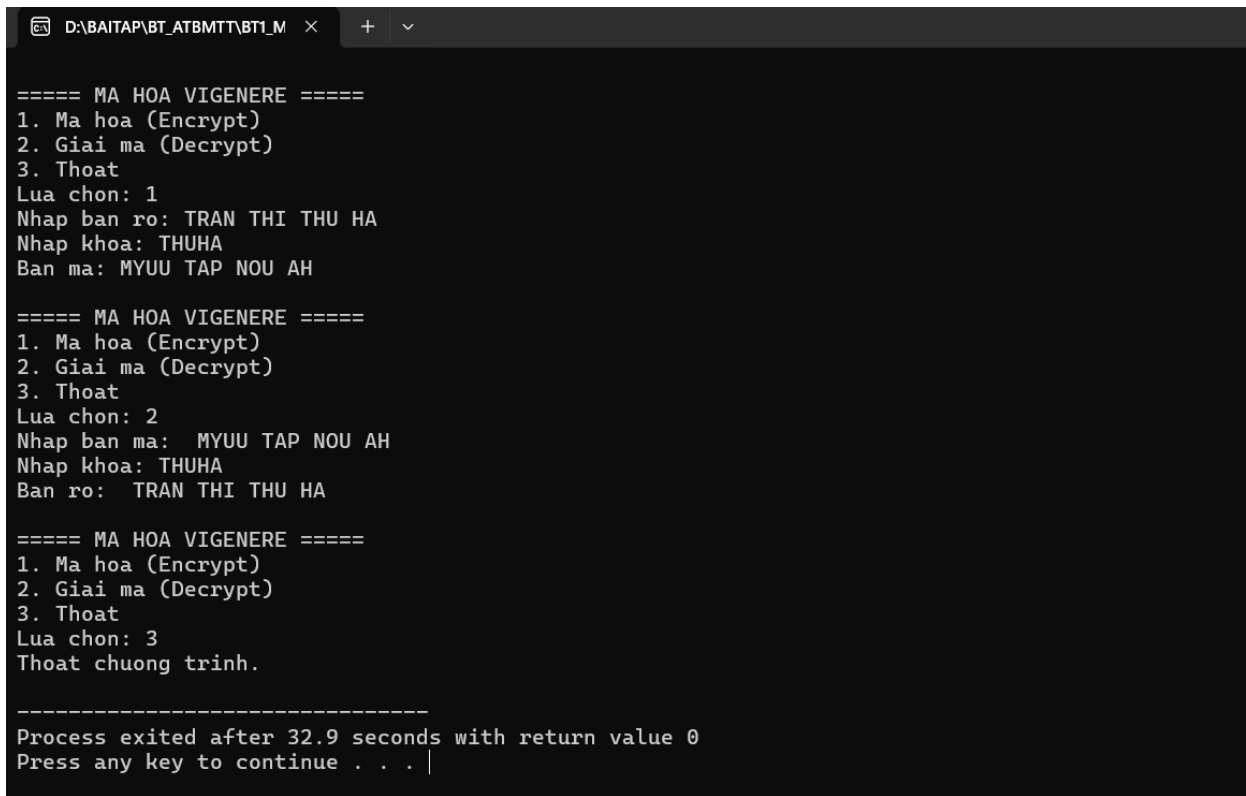
Hình 2.11. Mã hóa Vigenere

Giải mã bằng HTML + CSS + Javascript:



Hình 2.12. Giải mã thuật toán mã hóa Vigenere

Mã hóa và giải mã bằng C++:



```
D:\BAITAP\BT_ATBMTT\BT1_M x + v
===== MA HOA VIGENERE =====
1. Ma hoa (Encrypt)
2. Giai ma (Decrypt)
3. Thoat
Lua chon: 1
Nhap ban ro: TRAN THI THU HA
Nhap khoa: THUHA
Ban ma: MYUU TAP NOU AH

===== MA HOA VIGENERE =====
1. Ma hoa (Encrypt)
2. Giai ma (Decrypt)
3. Thoat
Lua chon: 2
Nhap ban ma: MYUU TAP NOU AH
Nhap khoa: THUHA
Ban ro: TRAN THI THU HA

===== MA HOA VIGENERE =====
1. Ma hoa (Encrypt)
2. Giai ma (Decrypt)
3. Thoat
Lua chon: 3
Thoat chuong trinh.

-----
Process exited after 32.9 seconds with return value 0
Press any key to continue . . . |
```

Hình 2.13. Mã hóa và giải mã thuật toán Vigenere bằng C++

2.5. Phương pháp mã hóa PlayFair

2.5.1. Tên gọi

Mã hóa PlayFair Cipher là phương pháp mã hóa theo cặp chữ

2.5.2. Thuật toán mã hóa và giải mã

- Tạo một mã trận 5x5 từ khóa, viết khóa (loại bỏ chữ trùng lặp) rồi điền các chữ cái còn lại theo thứ tự bảng chữ cái.
- Quy ước phổ biến gộp I và J thành một ô (thay J bằng I).

Thuật toán mã hóa:

Với mỗi digraph(A, B):

- Nếu A và B nằm cùng hàng trong ma trận thì thay bằng chữ cái ở bên phải (dịch phải).
- Nếu A và B cùng cột thì thay bằng chữ cái ở bên dưới (dịch xuống).
- Nếu A và B tạo thành hai góc của một hình chữ nhật thì thay mỗi chữ bằng chữ cùng hàng nhưng cột tương ứng với cột của chữ kia.

Thuật toán giải mã:

Làm ngược lại với thuật toán mã hóa:

- Cùng hàng: thay bằng chữ ở bên trái
- Cùng cột: thay bằng chữ ở trên
- Hình chữ nhật: đổi cột tương tự quy tắc mã hóa nhưng làm ngược lại

2.5.3. Không gian khóa

Mã hóa PlayFair dùng 25 ô (ma trận 5x5) nên số cách sắp xếp là $25!$ $\Rightarrow$ không gian khóa rất lớn

2.5.4. Cách phá mã không cần khóa

- **Phân tích tần suất digraph:** thống kê tần suất các cặp chữ cái trong bản mã
- **Crib/Known-plaintext:** Nếu biết hoặc đoán một đoạn plaintext, dùng vị trí đó để kiểm tra cấu hình bảng khóa.
- **Phân tích cấu trúc cặp:** dò các quy tắc hình chữ nhật, hàng, cột để rút ra các quan hệ giữa các ký tự đã mã hoá $\rightarrow$ suy ra vị trí bảng.

2.5.5. Cài đặt thuật toán mã hóa và giải mã

Mã hóa bằng HTML + CSS + Javascript:

Thuật toán Playfair Cipher

Văn bản:

TRANHITHUHA

Khóa (chữ cái):

TRAN

Ma trận khóa 5x5:

T	R	A	N	B
C	D	E	F	G
H	I	K	L	M
O	P	Q	S	U
V	W	X	Y	Z

Mã hóa

Giải mã

Kết quả:

RANBCOHRMOKT

Hình 2.14. Mã hóa Playfair Cipher

Giải mã bằng HTML + CSS + Javascript:

Thuật toán Playfair Cipher

Văn bản:

RANBCOHRMOKT

Khóa (chữ cái):

TRAN

Ma trận khóa 5x5:

T	R	A	N	B
C	D	E	F	G
H	I	K	L	M
O	P	Q	S	U
V	W	X	Y	Z

Mã hóa

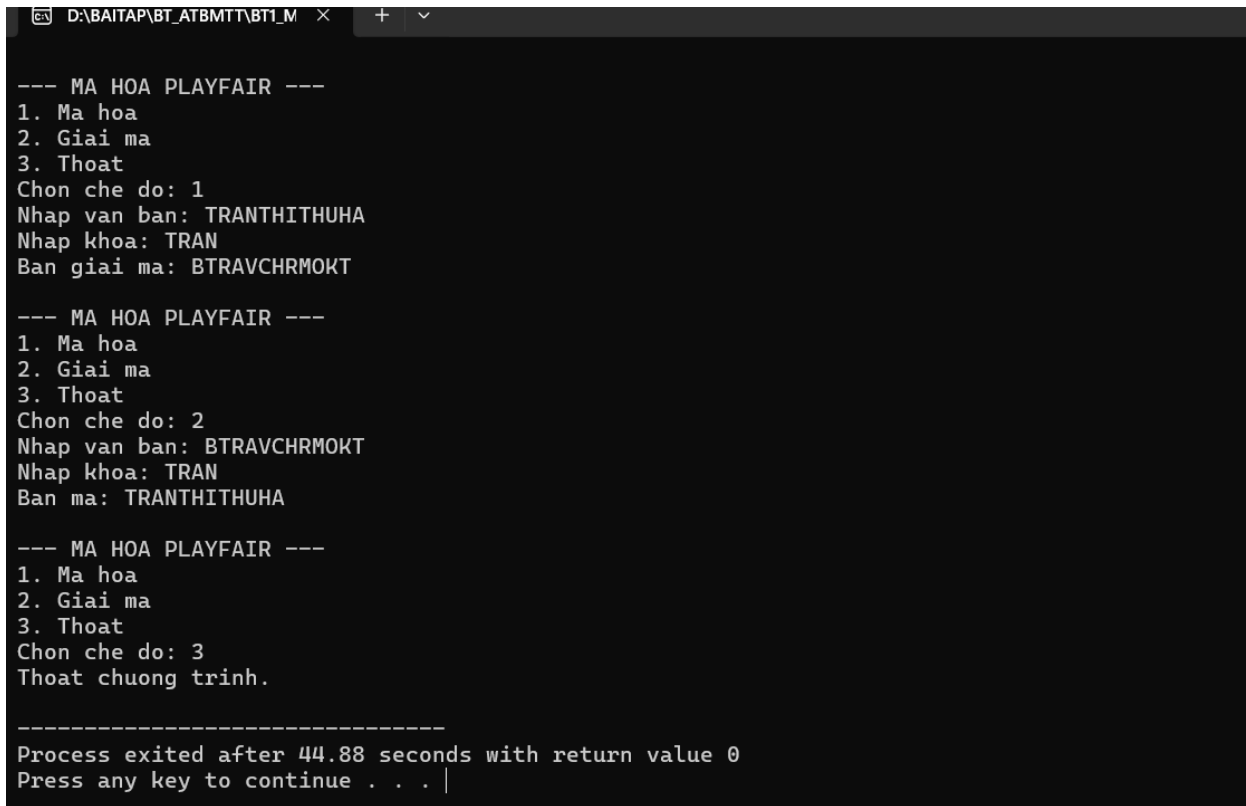
Giải mã

Kết quả:

TRANHITHUHA

Hình 2.15. Giải mã thuật toán PlayFair Cipher

Mã hóa và giải mã thuật toán Playfair Cipher bằng C++:



```
D:\BAITAP\BT_ATBMTT\BT1_M >
--- MA HOA PLAYFAIR ---
1. Ma hoa
2. Giai ma
3. Thoat
Chon che do: 1
Nhap van ban: TRANTHITHUHA
Nhap khoa: TRAN
Ban giai ma: BTRAVCHRMOKT

--- MA HOA PLAYFAIR ---
1. Ma hoa
2. Giai ma
3. Thoat
Chon che do: 2
Nhap van ban: BTRAVCHRMOKT
Nhap khoa: TRAN
Ban ma: TRANTHITHUHA

--- MA HOA PLAYFAIR ---
1. Ma hoa
2. Giai ma
3. Thoat
Chon che do: 3
Thoat chuong trinh.

-----
Process exited after 44.88 seconds with return value 0
Press any key to continue . . . |
```

Hình 2.16. Mã hóa và giải mã Playfair cipher bằng C++

CHƯƠNG 3. CHỮ KÝ SỐ TRONG FILE PDF

3.1. Cấu trúc file PDF liên quan chữ ký.

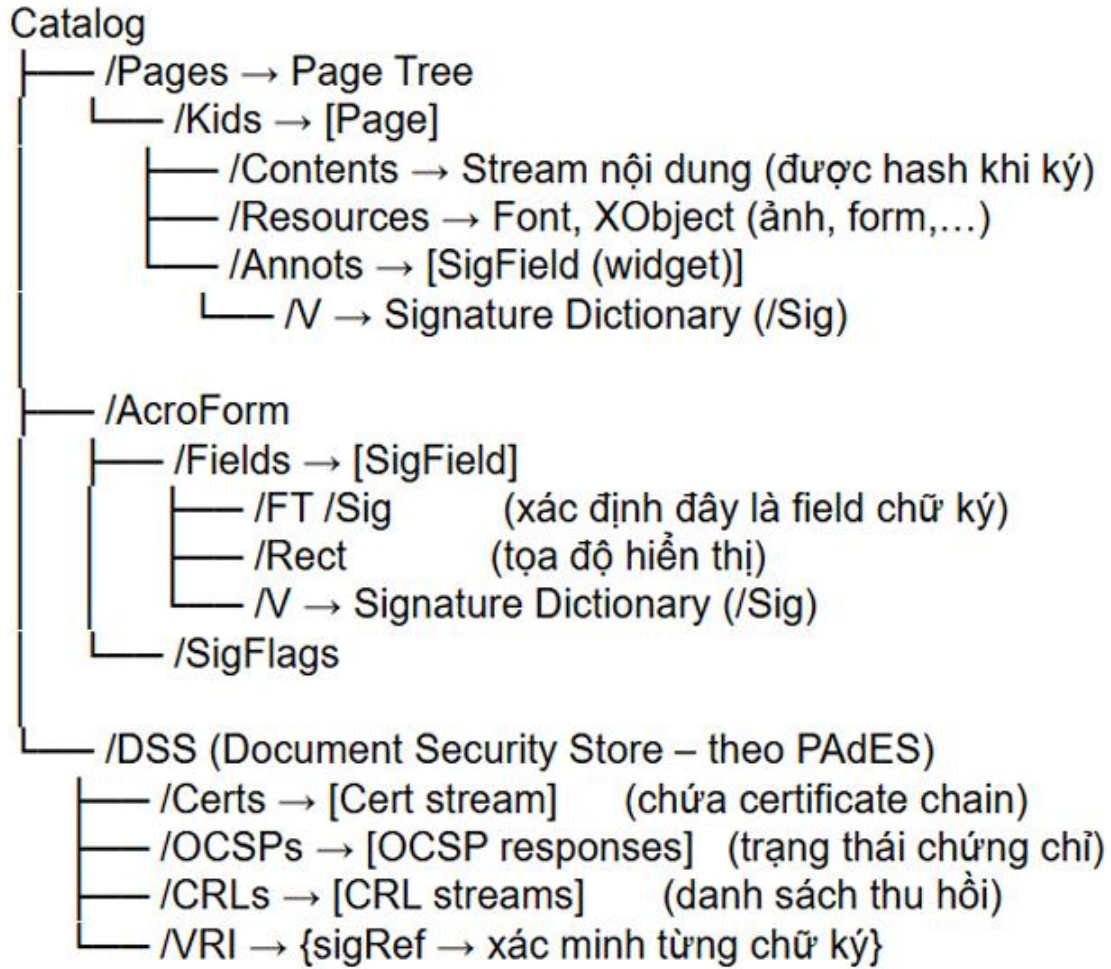
Mọi thành phần của file PDF được tổ chức dưới dạng object và liên kết với nhau bằng object reference. Chữ ký số trong PDF được lưu thông qua AcroForm, trong đó có Signature Field (Widget) để hiển thị chữ ký, Signature Dictionary(/Sig) để chứa thông tin chữ ký và dữ liệu mã hóa. Khi người dùng ký tài liệu, phần mềm PDF sẽ thực hiện incremental update là ghi thêm một lớp mới vào cuối file và giữ nguyên nội dung cũ để bảo đảm tính toàn vẹn, file còn có thêm DSS (Document Security Store) để lưu chứng thư phục vụ xác minh dài hạn.

Object refs quan trọng và vai trò:

Object Refs	Vai trò
Catalog	Là Root object chứa /AcroForm và /Pages. Quản lý toàn bộ cấu trúc file PDF, là điểm khởi đầu để truy suất đến SigField.
Pages tree	Cấu trúc cây trang, được tham chiếu từ Catalog. Tổ chức các Page object và Annots, trong đó có SigField widget.
Page object	Mô tả từng trang. Chứa /Contents (dòng lệnh vẽ nội dung trang) và /Resources (font, hình ảnh). Nếu có chữ ký hiển thị, vị trí của widget chữ ký được định nghĩa tại đây.
Resources	Dictionary chứa tài nguyên (font, XObject). Giúp render phần hiển thị chữ ký (ví dụ hình ảnh, text, logo của chữ ký).
Content streams	Stream chứa nội dung trang (text, vector, hình ảnh). Khi xác minh, phần này thuộc vùng /ByteRange để bảo vệ tính toàn vẹn nội dung.
XObject	External object (như hình ảnh). Thường được dùng để hiển thị hình ảnh chữ ký tay hoặc logo của người ký.
AcroForm	Form container nằm trong Catalog. Quản lý danh sách /Fields, trong đó có trường chữ ký (SigField).

Signature field (wifdget)	Annotation trên trang, biểu diễn khung chữ ký. Thuộc loại /Widget, có /FT /Sig. Xác định vị trí UI của chữ ký trên trang.
Signature dictionary	Lưu trữ dữ liệu chữ ký: /Contents (dạng PKCS#7), /ByteRange, /M (thời gian ký), /Name, /Reason, /Location. Là nơi thực hiện xác thực và validate hash.
ByteRange	Mảng [start1 length1 start2 length2]. Xác định phạm vi byte được ký (ngoại trừ vùng /Contents chứa chữ ký). Đảm bảo phát hiện thay đổi trái phép
Contents	Dữ liệu chữ ký số (PKCS#7/CMS) chứa hash, chứng chỉ, timestamp. Được giải mã để lấy thông tin người ký và xác minh tính toàn vẹn.
Incremental updates	Mỗi lần ký mới, PDF không bị ghi đè mà thêm phần “revision” mới ở cuối file. Giúp hỗ trợ ký nhiều lần (multi-signature) mà không phá vỡ chữ ký trước.
DSS	Document Security Store (chuẩn PAdES). Nằm trong Catalog, lưu trữ chứng chỉ (/Certs), phản hồi OCSP (/OCSPs), và danh sách CRL (/CRLs). Hỗ trợ xác minh chữ ký dài hạn (LTV)

Sơ đồ objects:



Hình 3.1. Sơ đồ objects

3.2. Nơi lưu thời gian ký

Tất cả các vị trí có thể lưu thông tin thời gian:

Vị trí	Miêu tả	Kiểu dữ liệu	Vai trò/Giá trị pháp lý
/M (Modification time) Vai trò/Giá	Thuộc tính /M nằm trong object /Sig	Chuỗi text định dạng pdf date	Là thời gian người ký khai báo tại thời

trị pháp lý trong Signature Dictionary			điểm ký, không được bảo đảm bởi bên thứ ba nên không có giá trị pháp lý, chỉ mang tính thông tin
Timestamp Token (RFC 3161) trong PKCS#7	Là attribute timeStamp Token bên trong chữ ký PKCS#7 (CMS) trong trường /content	Mã hóa dạng binary DER (theo RFC 3161)	Là bằng chứng thời gian được cung cấp bởi Time Stamping Authority (TSA). Có giá trị pháp lý vì được ký bằng chứng thư của TSA.
Document TimeStamp Object (PAdES)	Được thêm như signature riêng biệt trong file PDF, dùng filter /ETSI.RFC3161	Một loại Signature Dictionary đặc biệt	Được dùng để đóng dấu thời gian cho toàn bộ tài liệu (document-level timestamp). Có giá trị pháp lý
DSS (Document Security Store)	Lưu chữ dữ liệu hỗ trợ xác minh lâu dài: chứng thư, OCSP, CRL và timestamp token	Dictionary (theo chuẩn PAdES Part 4)	Không trực tiếp ký nhưng giữ lại timestamp token để xác thực trong tương lai khi TSA hoặc CA không còn hoạt động

Sự khác biệt giữa thông tin thời gian /M và timestamp RFC3161:

- **/M (Metadata time):** nơi lưu là trong Signature Dictionary (/Sig), dạng dữ liệu là chuỗi text (PDF date string) do phần mềm ký chèn vào, cách tạo là người ký hoặc phần mềm tự ghi, không đảm bảo và có thể giả mạo, không có giá trị pháp lý, công dụng chính là hiển thị thời gian ký.
- **Timestamp RFC3161:** nơi lưu là trong PKCS#7 (CMS) → SignedAttributes → timeStampToken, dạng dữ liệu là binary DER-encoded (ASN.1 structure) do TSA (time stamping authority) phát hành, cách tạo là gửi hash đến TSA và TSA ký rồi trả về token, được ký bằng khóa riêng của TSA nên được xác thực, có giá trị pháp lý, chống giả mạo (bằng chứng thời gian độc lập), công dụng chính là chứng minh tài liệu tồn tại trước một thời điểm cụ thể.

3.3. Các rủi ro bảo mật trong chữ ký số PDF

- **Padding Oracle Attack (tấn công đệm RSA):** Do lỗ hổng trong cơ chế RSA PKCS#1 v1.5, kẻ tấn công có thể khai thác phản hồi lỗi để đoán khóa, gây ra hậu quả có thể giả mạo chữ ký hoặc suy ra khóa riêng. Biện pháp dùng RSA-PSS, cập nhật thư viện crypto, không hiển thị lỗi chi tiết.
- **Lộ private key (key leak):** do khóa riêng bị sao chép hoặc đánh cắp (file .pem không mã hóa, malware,...), gây ra hậu quả kẻ tấn công ký giả mạo mọi tài liệu, phòng chống bằng cách lưu khóa trong HSM/USB token, mã hóa bằng passphrase và giới hạn quyền truy cập.
- **Replay Attack:** Tái sử dụng blob#7 hợp lệ trên tài liệu khác nếu không kiểm tra ByteRange gây ra hậu quả tài liệu vẫn có chữ ký hợp lệ. Phòng chống: kiểm tra /ByteRange và hash, dùng **RFC3161 timestamp**, ghi log.
- **Signature Wrapping:** Kẻ tấn công có thể di chuyển hoặc nhân bản object chữ ký sang vị trí khác trong PDF. Gây ra hậu quả nội dung hiển thị khác với phần thực sự được ký. Phòng chống bằng cách kiểm tra vị trí SigDict, xác minh ByteRange, khóa nội dung sau ký.

- **Incremental Update Injection:** Kẻ tấn công có thể thêm update mới sau chữ ký (append) để thay đổi nội dung hiển thị, gây ra hậu quả là hai tài liệu khác nhau có cùng hash → chữ ký “hợp lệ” cho cả hai. Phòng chống bằng cách dùng **SHA-256 / SHA-512**, từ chối SHA-1/MD5.
- **Chứng thư kiểm tra không đầy đủ (CRL/OCPS thiếu):** Không kiểm tra trạng thái thu hồi hoặc EKU của certificate. Hậu quả là chấp nhận cert đã bị thu hồi/hết hạn. Phòng chống bằng cách kiểm tra **OCSP/CRL**, xác minh EKU, lưu kết quả trong **DSS (PAdES-LTV)**.
- **Timestamp giải hoặc thiếu:** Chỉ có /M (local time) hoặc không có RFC3161 từ TSA, hậu quả là không xác minh được thời điểm ký và dễ lách hạn cert. Phòng chống bằng cách sử dụng RFC3161 timestamp token từ TSA tin cậy và lưu token trong PKCS#7/DSS.

3.4. Quy trình tạo và lưu chữ ký số trong file PDF

Bước 1: Chuẩn bị file PDF gốc

Chọn file PDF cần ký, ví dụ: original.pdf. File này có thể chứa văn bản, hình ảnh hoặc form. Mục tiêu là thêm trường chữ ký (Signature Field) mà không làm thay đổi nội dung gốc.

Bước 2: Tạo trường chữ ký (AcroForm / Signature Field)

Tạo AcroForm Dictionary trong file PDF nếu chưa có, thêm Signature Field (/SigField) để chứa chữ ký. Reserve vùng /Contents (8192 bytes) để chèn chữ ký nhị phân.

Bước 3: Xác định vùng /ByteRange

ByteRange là danh sách 4 số [start1, length1, start2, length2], chỉ định vùng dữ liệu được hash. Vùng /Contents được loại trừ để có thể chèn chữ ký sau.

Bước 4: Tính hash trên vùng ByteRange

Đọc hai vùng dữ liệu theo ByteRange, nối lại và tính giá trị băm SHA-256 hoặc SHA-512.

Bước 5: Tạo cấu trúc PKCS#7 (CMS Detached Signature)

Tạo chữ ký theo chuẩn PKCS#7 gồm các thành phần: messageDigest, signingTime, contentType, certificate chain, signerInfo và tùy chọn timestamp RFC3161.

Bước 6: Chèn chữ ký vào vùng /Contents

Sau khi có chữ ký PKCS#7 dạng DER, chèn vào /Contents dưới dạng hex. Chiều dài phải khớp với vùng dự trữ (8192 bytes).

Bước 7: Ghi Incremental Update

Thực hiện lưu file PDF theo dạng incremental (không ghi đè), giúp bảo toàn nội dung và hỗ trợ nhiều chữ ký.

Bước 8: Cập nhật DSS (Document Security Store - LTV)

Thêm trường /DSS vào Catalog chứa Certs, OCSP, CRL, VRI để đảm bảo xác thực lâu dài (Long-Term Validation).

Bước 9: Thông số kỹ thuật

- Hash algorithm: SHA-256
- RSA key size: 2048 bits
- Padding: PKCS#1 v1.5
- Signature type: CMS detached (CAdES-BES)
- Encoding: DER (hex trong /Contents).

Bước 10: Kết quả đầu ra

- Mã nguồn Python ký PDF (pyHanko, pikepdf, cryptography).
- File PDF gốc: original.pdf
- File PDF đã ký: signed_output.pdf
- Có thể xác thực bằng Adobe Acrobat hoặc script verify_pdf_signature.py

Sơ đồ quy trình: original.pdf → Thêm AcroForm → Tính hash → Tạo PKCS#7 → Ghi vào /Contents → Lưu incremental update → Thêm DSS → signed_output.pdf

Kết luận: Quy trình tuân thủ chuẩn ETSI EN 319 142-1 (CAAdES) và ISO 32000-2 (PDF 2.0), hỗ trợ mở rộng timestamp, OCSP, CRL để tăng tính pháp lý.

3.4.1. Chuẩn bị một file PDF gốc cần ký

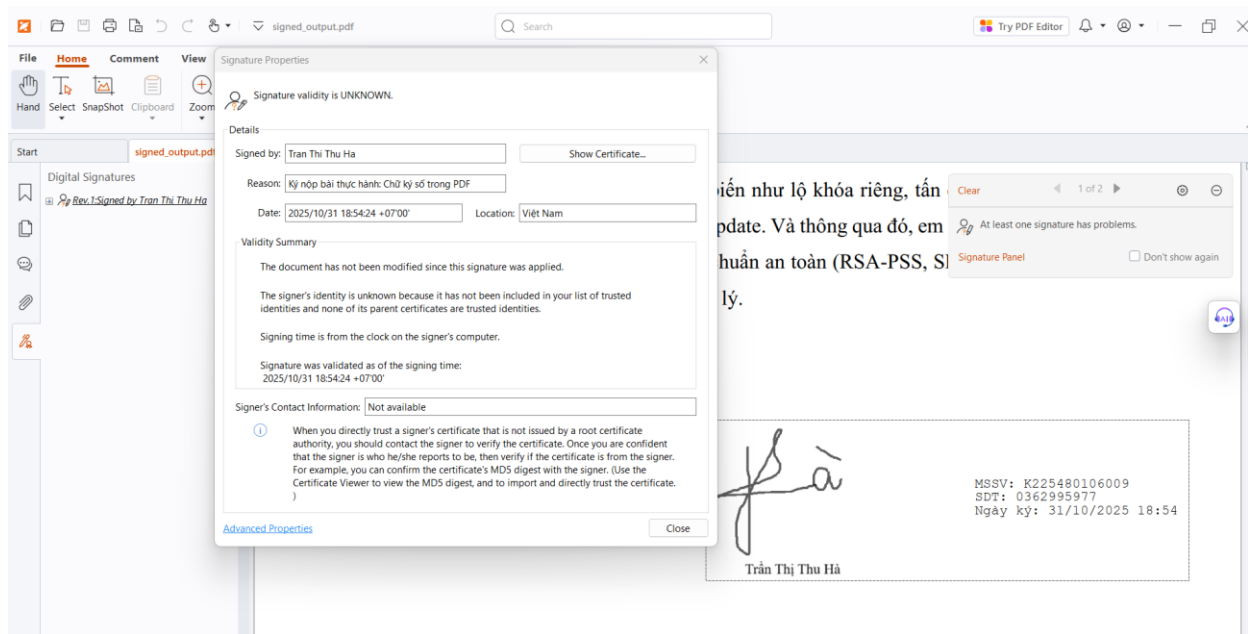
File: *original.pdf*

3.4.2. Tạo file sinh khóa RSA và chứng thư số

Sinh khóa RSA và chứng thư số mục đích là để xác thực chữ ký số và ký. Tạo một file *genn_key.py*, sau khi chạy file này sẽ sinh ra hai file *signer_cert.pem* (chứng thư) và file *signer_key.pem* (private key).

3.4.3. Tạo chữ ký và ký file PDF

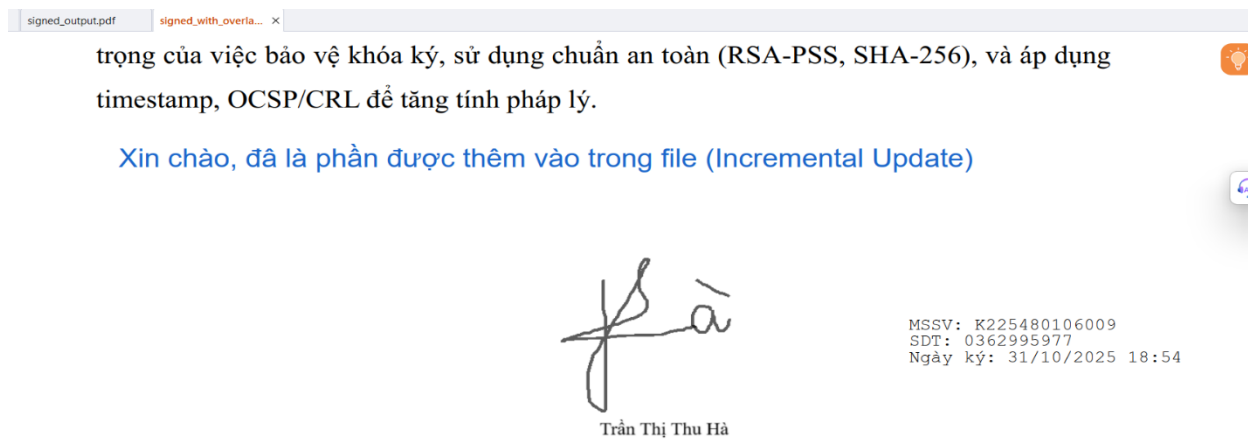
- Ta sẽ thực hiện tạo Signature field (AcroForm) và reserve vùng /Contents (8192 bytes), xác định /ByteRange để loại trừ vùng /Contents khỏi hash) và tính Hash (SHA-256) trên vùng ByteRange. Sinh PKCS#7 detached signature và chèn blob DER PKCS#7 vào /Contents đúng offset bằng file *sign_pdf.py*.
- Sau khi chạy lệnh *python sign_pdf.py* thì các bước trên đã được thực hiện và tạo ra một file mới đã được ký chữ ký số hợp lệ *signed_output.pdf*
- Đây là kết quả sau khi chạy file *sign_pdf.py*: tạo ra một file tên *signed_output.pdf* có chữ ký số



Hình 3.2. Kết quả file pdf đã được ký

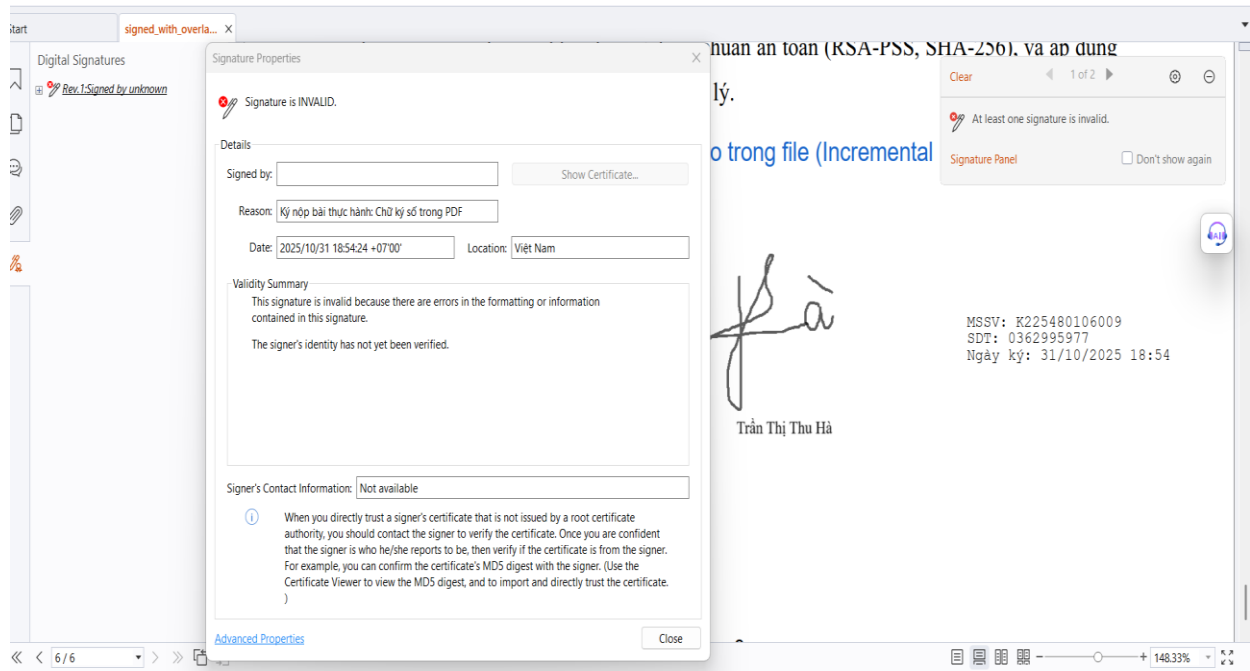
Chèn thêm nội dung vào file PDF:

- Tạo một file `update_pdf_incremental.py` sau đó thêm một đoạn nội dung vào file đã được ký trước đó, ví dụ “Xin chào”, sau đó chạy lệnh `python update_pdf_incremental.py` sẽ tạo ra một file pdf đã được chèn thêm nội dung vào (`signed_with_overlay.pdf`).



Hình 3.3. File pdf đã ký được chèn thêm nội dung vào

Đây là thông báo từ kết quả xác thực đã khác đi khi thêm nội dung vào file



Hình 3.4. Kết quả thông báo xác thực của file pdf bị chèn thêm nội dung

3.5. Xác thực chữ ký trên file PDF đã ký

Các bước xác thực chữ ký trên file PDF:

- Đọc Signature dictionary: /Contents, /ByteRange.
- Tách PKCS#7, kiểm tra định dạng.
- Tính hash và so sánh messageDigest.
- Verify signature bằng public key trong cert.
- Kiểm tra chain → root trusted CA.
- Kiểm tra OCSP/CRL.
- Kiểm tra timestamp token.
- Kiểm tra incremental update (phát hiện sửa đổi). Tạo một file *verify_pdf_signature.py*, file này sẽ thực hiện các bước ở trên và khi thực hiện chạy

python verify_pdf_signature.py sẽ tạo ra file xác minh chữ ký trong file *signed_output.pdf* có hợp lệ hay không.

Kết quả:

- Tạo ra file *verify_KQ_ok.txt* khi kiểm tra file *signed_output.pdf* có chữ ký hợp lệ.

```
Thông tin chứng thư người ký:
Chủ thể: Common Name: Tran Thi Thu Ha, Organization: 58KTP, Locality: Thai Nguyen, State/Province: Thai Nguyen, Country: VN
SHA1 fingerprint: FF CA 01 C3 F2 DE B4 E6 AF 5E DB 4C 97 D6 9D 49 DF 42 12 7A
SHA256 fingerprint: EC 5D 89 A4 FB D2 08 B9 DE FE 71 8D E8 AB E2 28 2D D3 92 96 19 07 F3 21 85 13 9 A 5B A0 14 7C 0E
🕒 Thời gian ký (VN): 2025-10-31 18:54:24+07:00
✅ File chưa bị chỉnh sửa kể từ khi ký.
=====
✅ KẾT LUẬN: CHỮ KÝ TỰ KÝ (SELF-SIGNED) - FILE NGUYÊN VẸN.
⚠️ Lưu ý: Chứng thư không thuộc CA tin cậy, nhưng dữ liệu không bị thay đổi.
📄 Log đã lưu tại: D:\BAITAP\BT_ATBMTT\BT2_CHUKYSO_SECURITY\Scripts\verify_KQ_ok.txt
```

Hình 3.5. Kết quả kiểm tra file PDF có chữ ký hợp lệ

- Tạo ra file *verify_file_overlay.txt* khi kiểm tra file *signed_with_overlay.pdf* đã được chèn thêm nội dung.

```
Thông tin chứng thư người ký:
Chủ thể: Common Name: Tran Thi Thu Ha, Organization: 58KTP, Locality: Thai Nguyen, State/Province: Thai Nguyen, Country: VN
SHA1 fingerprint: FF CA 01 C3 F2 DE B4 E6 AF 5E DB 4C 97 D6 9D 49 DF 42 12 7A
SHA256 fingerprint: EC 5D 89 A4 FB D2 08 B9 DE FE 71 8D E8 AB E2 28 2D D3 92 96 19 07 F3 21 85 13 9 A 5B A0 14 7C 0E
🕒 Thời gian ký (VN): 2025-10-31 18:54:24+07:00
❌ File đã bị thay đổi sau khi ký!
=====
❌ CHỮ KÝ KHÔNG HỢP LỆ hoặc FILE BỊ SỬA ĐỔI.
⚠️ Lý do thường gặp: chứng thư tự ký (self-signed), không có nonRepudiation, hoặc không thuộc CA tin cậy.
📄 Log đã lưu tại: D:\BAITAP\BT_ATBMTT\BT2_CHUKYSO_SECURITY\Scripts\verify_file_overlay.txt
```

Hình 3.6. Kết quả kiểm tra file PDF đã được chèn thêm nội dung

3.6. Kết quả demo

Sau khi thực hiện các yêu cầu của đề bài, tạo ra các file:

- Original.pdf: đây là file gốc chưa có chữ ký
- Signed_output.pdf: là file đã được ký và chứa chữ ký số hợp lệ
- Signed_with_overlay.pdf: là file chứa nội dung bị thay đổi sau khi đã ký chữ ký hợp lệ

- Verify_file_overlay.txt: là file chứa kết quả xác minh không hợp lệ do nội dung đã bị thay đổi.
- Verify_KQ_ok.txt: là file chứa kết quả xác minh hợp lệ.

TÀI LIỆU THAM KHẢO

1. Giáo trình An toàn và Bảo mật thông tin:

<https://luanvan.net.vn/luan-van/giao-trinh-an-toan-va-bao-mat-thong-tin-45788/>

2. Giáo trình lý thuyết Mật mã và An toàn thông tin – Phan Đình Diệu:

<https://tailieusinhvien.net/giao-trinh-ly-thuyet-mat-ma-va-an-toan-thong-tin-phan-dinh-dieu-1529/>

3. Tài liệu An toàn và Bảo mật thông tin – Download.vn: <https://www.zun.vn/tai-lieu/an-toan-bao-mat-thong-tin-mat-ma-co-dien-42017/>

4. Sercurity of PDF Signatures: https://www.pdf-insecurity.org/download/DIGITALVERSION_KMeyerZuSelhausen_SecurityOfPDFSignatures_2018-11-25.pdf

LINK GITHUB

1. Link: https://github.com/ThuHa2004/BTL_ANTOAN-BMTT.git

2. Mã QR:

